Università degli Studi di Padova

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica



Progettazione e sviluppo di un Relying Party conforme allo standard EUDI Wallet: dal parsing di query DCQL alla convalida crittografica

Tesi di laurea triennale

Relatore

Prof. Giovanni Da San Martino

Laureando

Andrea Masiero

Anno Accademico 2025-2026

Sommario

Il presente documento descrive l'attività svolta dal laureando Andrea Masiero durante il periodo di stage presso l'azienda Monokee. L'esperienza, svolta a conclusione del corso di laurea triennale in Informatica, ha avuto una durata complessiva di trecentoventi ore.

Il lavoro di tesi illustra la progettazione e lo sviluppo di un *Relying Party* in ambiente Node.js/TypeScript, conforme alle specifiche dell'ecosistema EUDI Wallet, con particolare attenzione al protocollo OpenID for Verifiable Presentations (OpenID4VP). Il progetto copre l'intero ciclo di vita dell'interazione con il Wallet, focalizzandosi in particolare su due aspetti tecnici avanzati: le richieste dinamiche e la validazione crittografica. Da un lato, il sistema implementa il parsing e l'elaborazione del Dynamic Credential Query Language (DCQL), permettendo al *Relying Party* di formulare richieste complesse, flessibili e multi-credenziale, superando i limiti delle query statiche. Dall'altro, il nucleo del progetto si concentra sulla rigorosa verifica delle credenziali, supportando nativamente il formato SD-JWT (Selective Disclosure JWT). Il risultato finale è un componente architetturale robusto, modulare e pronto per essere integrato in scenari reali di identità digitale, dimostrando la fattibilità tecnica e l'efficacia dei nuovi standard europei per lo scambio sicuro di credenziali verificabili.

Indice

1 Il contesto aziendale **1**
- 1.1 Scelta dello stage . . . 1
- 1.2 Dominio applicativo . . . 1
- 1.3 Tecnologie utilizzate . . . 2
  - 1.3.1 TypeScript . . . 2
  - 1.3.2 Node.js . . . 2
  - 1.3.3 Express.js . . . 2
  - 1.3.4 MongoDB . . . 2
  - 1.3.5 Redis . . . 3
  - 1.3.6 jose (JavaScript Object Signing and Encryption) . . . 3
  - 1.3.7 Docker . . . 3
  - 1.3.8 ESLint . . . 3
  - 1.3.9 Prettier . . . 3
  - 1.3.10 Yarn . . . 3
  - 1.3.11 Nodemon . . . 4
  - 1.3.12 dotenv . . . 4
  - 1.3.13 Librerie Interne Monokee . . . 4
  - 1.3.14 Azure Container Registry (ACR) . . . 5

2 Analisi dei Requisiti **7**
- 2.1 Panoramica e Attori . . . 7
  - 2.1.1 Attori del sistema . . . 7
  - 2.1.2 Requisiti Obbligatori . . . 7
  - 2.1.3 Requisiti Desiderabili . . . 8
- 2.2 Casi d'uso . . . 8
- 2.3 Limiti . . . 21

3 Progettazione **23**
- 3.1 Architettura . . . 23

4 Codifica **25**
- 4.1 Organizzazione del codice . . . 25

5 Verifica e validazione **27**

6 Valutazione finale **29**
- 6.1 Risultati ottenuti . . . 29
- 6.2 Analisi critica del prodotto . . . 29
- 6.3 Analisi critica del lavoro . . . 29
- 6.4 Valutazione degli strumenti utilizzati . . . 31

Bibliografia **33**

Elenco delle figure

1.1 Logo di Monokee 1

2.1 Diagramma dei casi d'uso – Scenario principale del Relying Party 9
2.2 UC1 – Inizializzazione della richiesta di presentazione 9
2.3 UC2 – Generazione del Request Object 14

Elenco delle tabelle

Capitolo 1

Il contesto aziendale

1.1 Scelta dello stage

L'opportunità di svolgere lo stage aziendale è scaturita dalla partecipazione all'evento *Stage-IT 2026*, un'iniziativa promossa per favorire il confronto tra gli studenti e diverse realtà aziendali operanti in svariati ambiti del *software development*. Tra le diverse proposte presentate, l'attenzione si è focalizzata sul progetto offerto dall'azienda *Monokee*, incentrato sui temi dell'identità digitale e delle credenziali decentralizzate. Tale scelta è stata dettata da fattori chiave distinti: da un lato, il vivo interesse per gli aspetti innovativi e le sfide tecnologiche del progetto; dall'altro, la stretta coerenza con gli obiettivi accademici e professionali futuri, orientati al proseguimento degli studi nella laurea magistrale in cybersicurezza. L'analisi della documentazione tecnica preliminare fornita dall'azienda, unitamente ai colloqui conoscitivi effettuati, ha permesso di delineare nel dettaglio gli obiettivi del progetto, consolidando la decisione di svolgere il percorso di tirocinio presso Monokee.

Figura 1.1: Logo di Monokee
Fonte: www.monokee.com

1.2 Dominio applicativo

Monokee implementa una piattaforma avanzata focalizzata sui sistemi di *Identity and Access Management* (IAM) e sull'orchestrazione delle identità digitali. La soluzione aziendale propone un approccio innovativo che integra la gestione dell'identità con un orchestratore visivo *low-code/no-code*, consentendo alle organizzazioni di modellare, automatizzare e personalizzare flussi complessi di autenticazione e autorizzazione per utenti e dispositivi (*Identity Journeys*) senza la necessità di sviluppare codice ad hoc.

All'interno di questa architettura tecnologica, l'attività di stage si è focalizzata sulla progettazione e sullo sviluppo *ex novo* del modulo `monokee-relying-party`. Tale componente consente alla piattaforma Monokee di operare come *Relying Party* (o *Verifier*) nell'ambito del paradigma delle credenziali decentralizzate e dell'ecosistema EUDI Wallet. Il modulo ha il compito di richiedere, ricevere e verificare le *Verifiable Credentials* presentate dai *wallet*

digitali degli utenti, convertendole in sessioni di accesso sicure e integrate nell'infrastruttura IAM aziendale.

Nello specifico, il componente si articola nelle seguenti funzionalità principali:

1. **Gestione delle *Presentation Requests*:** generazione e invio di richieste di presentazione standardizzate volte ad attestare specifici attributi dell'utente (ad esempio la maggiore età o la cittadinanza);

2. **Implementazione del protocollo OpenID4VP:** gestione completa del protocollo *OpenID for Verifiable Presentations* per la ricezione e l'elaborazione sicura dei *Verifiable Presentation Tokens* (*VP Tokens*) tramite *endpoint* dedicati;

3. **Validazione delle prove crittografiche:** verifica dell'integrità e dell'autenticità delle attestazioni ricevute — comprese quelle strutturate secondo lo standard *Selective Disclosure JWT* (SD-JWT) — e convalida della catena di fiducia (*Trust Chain*) legata agli emettitori (*Issuers*).

1.3 Tecnologie utilizzate

1.3.1 TypeScript

TypeScript è un linguaggio di programmazione open-source sviluppato da Microsoft che estende JavaScript aggiungendo la tipizzazione statica opzionale e funzionalità avanzate come le interfacce, gli enum e i generics. TypeScript è stato scelto come linguaggio principale per lo sviluppo del microservizio, garantendo maggiore robustezza del codice, migliore esperienza di sviluppo grazie all'autocompletamento e al controllo dei tipi a tempo di compilazione, e una manutenibilità superiore rispetto a JavaScript puro.

1.3.2 Node.js

Node.js è un runtime JavaScript lato server che consente l'esecuzione di codice al di fuori del browser. Il progetto utilizza Node.js 24 come ambiente di esecuzione, sfruttandone il modello di I/O non bloccante e ad eventi per gestire in modo efficiente le richieste HTTP concorrenti. Node.js rappresenta il fondamento su cui il microservizio viene costruito e distribuito, offrendo un ecosistema maturo e performante per applicazioni server-side.

1.3.3 Express.js

Express.js è un framework web minimalista e flessibile per Node.js, ampiamente adottato come standard de facto per la creazione di applicazioni server e API RESTful. Nel progetto viene utilizzato come base per il server HTTP, gestendo il routing delle richieste, i middleware di elaborazione (pre e post processing) e la gestione degli errori. Il framework è integrato tramite le librerie proprietarie Monokee (monokee-base-server e monokee-scim-utils) che ne estendono le funzionalità per adattarlo alle esigenze specifiche della piattaforma.

1.3.4 MongoDB

MongoDB è un database NoSQL orientato ai documenti, che memorizza i dati in formato Binary JSON, offrendo flessibilità nello schema e scalabilità orizzontale nativa. Nel progetto viene utilizzato come database principale per la persistenza delle configurazioni dei client (Verifier), attraverso il repository MongoDBClientRepository che estende la classe base MongoDBBaseRepository fornita dalla libreria interna @monokee-dev/monokee-base-repository. MongoDB è stato scelto per la sua capacità di gestire documenti complessi e strutturati in modo naturale, come le configurazioni crittografiche (JWKS) dei Verifier OpenID4VP.

1.3.5 Redis

Redis è un data store in-memory open-source, utilizzato come database chiave-valore ad alte prestazioni, cache e message broker. Nel progetto viene impiegato tramite la libreria ioredis per la gestione delle sessioni di verifica con supporto a TTL (Time-To-Live) configurabile. La scelta di Redis è motivata dalla necessità di avere uno storage veloce e volatile per dati temporanei come le sessioni di presentazione.

1.3.6 jose (JavaScript Object Signing and Encryption)

jose è una libreria JavaScript/TypeScript per la gestione dei JSON Web Token (JWT), JSON Web Signature (JWS), JSON Web Encryption (JWE) e JSON Web Key (JWK), conforme alle specifiche RFC. Nel progetto rappresenta una dipendenza fondamentale, utilizzata estensivamente per: la firma e la verifica di JWT (richieste di autorizzazione e Entity Statement), la cifratura e decifratura JWE delle risposte del wallet, l'importazione e gestione delle chiavi crittografiche (JWK/JWKS) per molteplici algoritmi (ES256, RS256, ECDH-ES, ecc.), e la validazione dei certificati X.509 nella trust chain OpenID Federation. La libreria è alla base di tutta la logica crittografica del microservizio.

1.3.7 Docker

Docker è una piattaforma di containerizzazione che consente di impacchettare un'applicazione e tutte le sue dipendenze in un contenitore leggero e portabile. Il progetto utilizza un Dockerfile multi-stage: il primo stage (builder) compila il codice TypeScript e installa le dipendenze, mentre il secondo stage copia solamente gli artefatti necessari all'esecuzione (dist, node_modules, resources), riducendo significativamente la dimensione dell'immagine finale.

1.3.8 ESLint

ESLint è uno strumento di analisi statica del codice (linter) per JavaScript e TypeScript che identifica pattern problematici, errori e violazioni delle convenzioni di stile. Nel progetto è configurato con il parser @typescript-eslint/parser e integra diversi plugin: @typescript-eslint per regole specifiche TypeScript (come le naming convention per classi PascalCase e metodi camelCase), eslint-plugin-security per l'identificazione di potenziali vulnerabilità di sicurezza, e eslint-plugin-prettier per l'integrazione con il formattatore di codice. ESLint è eseguito sia in fase di sviluppo locale (tramite Nodemon) sia nella pipeline CI/CD.

1.3.9 Prettier

Prettier è un formattatore di codice che impone uno stile di formattazione consistente in modo automatico, eliminando i dibattiti sullo stile del codice all'interno del team. Nel progetto è configurato con opzioni specifiche: senza punto e virgola (semi: false), con singoli apici (singleQuote: true), virgole finali ovunque (trailingComma: all) e parentesi obbligatorie nelle arrow function (arrowParens: always). Prettier è integrato con ESLint tramite eslint-config-prettier per evitare conflitti tra le regole dei due strumenti.

1.3.10 Yarn

Yarn è un package manager per JavaScript sviluppato originariamente da Facebook, alternativo a npm, che offre installazione più veloce delle dipendenze, un lockfile deterministico (yarn.lock) e una gestione più affidabile della risoluzione dei pacchetti. Nel progetto Yarn viene utilizzato per la gestione di tutte le dipendenze, sia nella fase di sviluppo locale sia nella build Docker, ed è configurato per accedere al registry npm privato di GitHub Packages dell'organizzazione Monokee tramite il file .npmrc.

1.3.11 Nodemon

Nodemon è uno strumento di sviluppo per Node.js che monitora le modifiche ai file sorgente e riavvia automaticamente l'applicazione. Nel progetto è configurato per osservare le directory src, il file .env e package.json, monitorando le estensioni .js, .ts, .json, .yml, .yaml e .env. Ad ogni modifica, Nodemon esegue automaticamente la sequenza completa: installazione dipendenze, build TypeScript, audit di sicurezza, lint del codice ed esecuzione dell'applicazione tramite ts-node, garantendo un ciclo di sviluppo rapido e completo.

1.3.12 dotenv

dotenv è una libreria che carica le variabili d'ambiente da un file .env nell'oggetto process.env di Node.js, consentendo la separazione della configurazione dal codice sorgente. Nel progetto viene utilizzato per gestire la configurazione dell'applicazione, come gli URL di connessione a MongoDB e Redis, il livello di logging, i parametri TTL delle sessioni, gli URL dell'endpoint di autorizzazione OpenID4VCI e le credenziali del Verifier (Entity Identifier e Verifier Attestation JWT). Questo approccio segue il principio del Twelve-Factor App per la gestione delle configurazioni.

1.3.13 Librerie Interne Monokee

Il progetto si appoggia su un ecosistema di librerie proprietarie sviluppate internamente dall'organizzazione Monokee, distribuite come pacchetti npm privati tramite GitHub Packages. Le principali librerie utilizzate sono:

- @monokee-dev/monokee-base-server: fornisce la classe Server che incapsula la configurazione del server Express.js, la gestione delle rotte, i middleware e l'Async Local Storage per il contesto delle richieste.

- @monokee-dev/monokee-base-repository: mette a disposizione MongoDBBaseRepository, una classe base generica per l'interazione con MongoDB che implementa il pattern Repository.

- @monokee-dev/monokee-base-service: fornisce classi base per la logica di business dei servizi.

- @monokee-dev/monokee-base-filter: implementa i meccanismi di filtraggio per le query sui repository.

- @monokee-dev/monokee-scim-utils: offre utilità per il protocollo SCIM (System for Cross-domain Identity Management), inclusa la classe ScimServer e ScimBaseRoute utilizzate come base per il server e le rotte del progetto.

- @monokee-dev/monokee-logger: fornisce MonokeeLogger, un wrapper di logging strutturato con supporto a livelli configurabili e contesto di autenticazione.

- @monokee-dev/monokee-utils: raccoglie utilità trasversali, tra cui la gestione degli errori tipizzati (ForbiddenError, InvalidValueError, NotFoundError, PreconditionFailedError) e l'Async Local Storage (MonokeeAlsStore).

- @monokee-dev/monokee-health-probe: implementa gli endpoint di health check per il monitoraggio dello stato del microservizio negli ambienti containerizzati (readiness/liveness probe).

1.3.14 Azure Container Registry (ACR)

Azure Container Registry è il servizio di registry per immagini Docker offerto da Microsoft Azure. Nel progetto viene utilizzato come destinazione per la pubblicazione delle immagini Docker prodotte dalla pipeline CI/CD. Le immagini vengono taggate con la data di build e un prefisso dell'hash del commit Git, e pubblicate sul registry monokee4prod.azurecr.io, pronte per essere distribuite negli ambienti di staging e produzione dell'infrastruttura cloud Monokee.

Capitolo 2

Analisi dei Requisiti

2.1 Panoramica e Attori

L'obiettivo dello stage è lo sviluppo di un modulo di relying party (come definito nella sezione 1.2) che si occuperà di creare e inviare richieste standardizzate per ottenere attributi specifici degli utenti, gestire la ricezione dei VP Tokens tramite endpoint dedicati e verificare l'integrità delle presentazioni e la validità degli emettitori.

2.1.1 Attori del sistema

Gli attori rappresentano le entità (utenti fisici o sistemi esterni) che interagiscono con l'applicazione. Per il progetto sviluppato, sono stati individuati i seguenti attori principali:

- **Relying Party:** rappresenta la componente backend del sistema che funge da entità richiedente e verificatrice delle Credenziali Verificabili. Gestisce l'intero ciclo di vita di una richiesta di presentazione: dall'inizializzazione della sessione, alla generazione del Request Object firmato crittograficamente, fino alla validazione delle presentazioni ricevute dal Wallet.
- **Wallet:** rappresenta un sistema esterno,tipicamente un'app mobile o un'estensione del browser, che custodisce le Credenziali Verificabili (Verifiable Credentials) emesse da uno o più Issuer fidati e le presenta su richiesta del Verifier. Nel contesto del protocollo OID4VP, il Wallet agisce come un intermediario tecnico tra l'Utente e il Relying Party: riceve la richiesta di presentazione, la interpreta, presenta all'Utente le credenziali richieste per ottenerne il consenso, e infine assembla e trasmette la Verifiable Presentation al Backend.
- **Utente finale:** è la persona fisica che interagisce con il sistema attraverso il Frontend del Relying Party e il proprio Wallet digitale. È il titolare delle Credenziali Verificabili, ovvero possiede documenti o attestazioni digitali (ad esempio un documento d'identità, una patente, un attestato professionale) emessi da Issuer fidati e memorizzati nel proprio Wallet. L'Utente rappresenta il soggetto che deve dimostrare il possesso di determinati attributi o qualifiche richieste dal servizio offerto dal Relying Party, ed è l'unico attore che può autorizzare la condivisione delle proprie credenziali.4

2.1.2 Requisiti Obbligatori

1. **O01**: Inizializzazione della sessione di presentazione
2. **O02**: Generazione dell'*Authorization Request URL*
3. **O03**: Esposizione dell'endpoint `request_uri`

4. **O04**: Pubblicazione dei metadati del Verifier (`client_metadata`)
5. **O05**: Prevenzione degli attacchi di tipo *replay*
6. **O06**: Ricezione della risposta del Wallet
7. **O07**: Validazione della struttura e dei criteri della *DCQL Query*
8. **O08**: Verifica della firma dell'Issuer (SD-JWT)
9. **O09**: Verifica del *Holder Binding* (KB-JWT)
10. **O10**: Recupero della presentazione verificata
11. **O11**: Gestione degli errori restituiti dal Wallet
12. **O12**: Gestione della configurazione del client Relying Party tramite SCIM

2.1.3 Requisiti Desiderabili

1. **D01**: Supporto alla modalità di risposta `direct_post.jwt` (JWE)
2. **D02**: Generazione di chiavi effimere per la cifratura JWE
3. **D03**: Risoluzione e verifica ricorsiva della *Trust Chain*
4. **D04**: Supporto a molteplici *Client ID Scheme* (`redirect_uri`, `x509_san_dns`, `x509_hash`, `openid_federation`, `verifier_attestation`, `pre_registered`)
5. **D05**: Esposizione dell'endpoint `/.well-known/openid-federation`

2.2 Casi d'uso

I diagrammi dei casi d'uso (*Use Case Diagram*) costituiscono una tipologia di diagrammi UML (*Unified Modelling Language*) finalizzata a descrivere le interazioni tra gli attori del sistema e le funzionalità che quest'ultimo è chiamato a fornire.

Nelle sottosezioni seguenti vengono descritti nel dettaglio i casi d'uso relativi all'integrazione del modulo Relying Party nel sistema. Ciascun caso d'uso è presentato secondo la seguente struttura standardizzata:

- **Caso d'uso:** codice identificativo e denominazione del caso d'uso;
- **Attori:** attori coinvolti nello scenario;
- **Descrizione:** descrizione dettagliata del caso d'uso, comprensiva dei passi che ne caratterizzano lo svolgimento;
- **Precondizioni:** condizioni che devono risultare soddisfatte affinché l'attore possa avviare il caso d'uso;
- **Postcondizioni:** stato del sistema al termine dell'esecuzione del caso d'uso;
- **Scenario principale:** sequenza ordinata di passi che descrivono il flusso nominale del caso d'uso, ossia la sequenza di azioni che si verifica in assenza di condizioni di errore;
- **Scenario alternativo:** eventuali flussi alternativi che possono verificarsi durante l'esecuzione del caso d'uso, con le relative condizioni di attivazione e le conseguenze sul sistema.

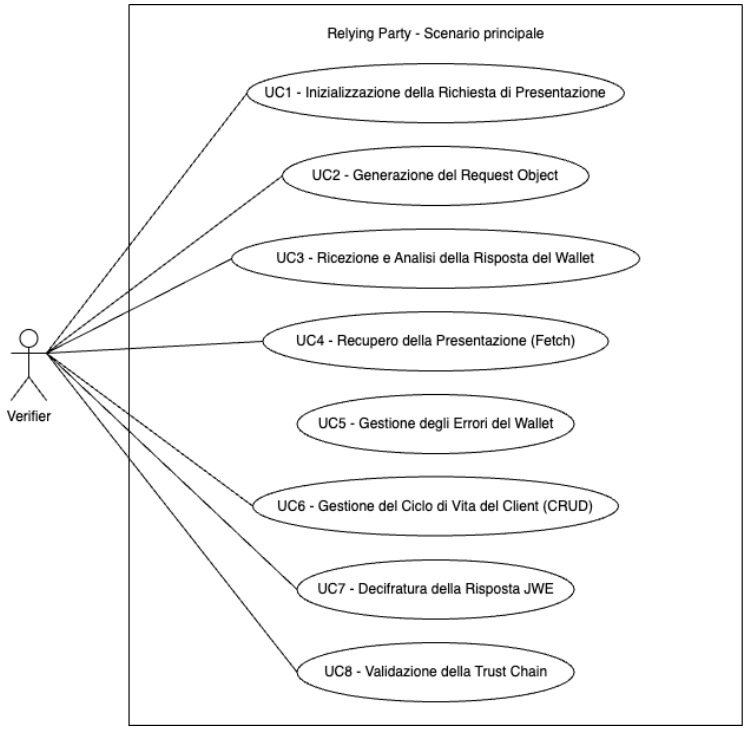


Figura 2.1: Diagramma dei casi d'uso – Scenario principale del Relying Party

UC1: Inizializzazione della richiesta di presentazione

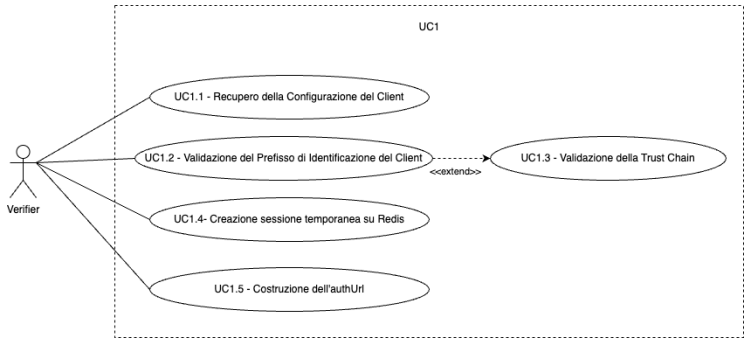


Figura 2.2: UC1 – Inizializzazione della richiesta di presentazione

Attori: Verifier, Utente finale.
Descrizione: L'Utente finale, mediante l'interfaccia *frontend* del Relying Party, avvia una nuova sessione di richiesta di presentazione di Credenziali Verificabili. Il Verifier procede alla validazione dell'identità del client precedentemente configurato, effettua — ove previsto dal *Client ID Scheme* adottato — la verifica della *Trust Chain* (nel caso di OpenID Federation), istanzia una sessione temporanea su Redis contenente la *DCQL Query*, un *nonce* crittografico e un identificativo univoco, e restituisce all'interfaccia *frontend* un URL di autorizzazione (`authUrl`) e un identificativo di sessione (`sessionId`).
Precondizioni:

- Il client (Verifier) è stato precedentemente configurato nell'istanza MongoDB con parametri `redirectUris` e `vpFormatsSupported` validi.

- Il server Redis è raggiungibile e operativo.
- Le variabili d'ambiente del sistema sono configurate correttamente.
- Qualora il prefisso del `clientId` sia `openid_federation`, la variabile d'ambiente `VERIFIER_ENTITY_IDENTIFIER` deve risultare impostata e le *Trusted Authorities* devono essere raggiungibili.
- Qualora il prefisso sia `verifier_attestation`, la variabile d'ambiente `VERIFIER_ATTESTATION_JWT` deve contenere un token JWT di attestazione valido.

Postcondizioni:

- Una nuova sessione con stato `pending` è stata istanziata su Redis con un TTL (*Time To Live*) configurato.
- La sessione contiene i seguenti parametri: `sessionId`, `requestId`, `nonce`, `transactionId`, `dcqlQuery`, `responseMode`, `prefixClientId`, `responseUri`.
- L'interfaccia *frontend* ha ricevuto l'`authUrl` (contenente le informazioni necessarie al Wallet per elaborare la richiesta) e il `sessionId` (necessario per il successivo recupero della presentazione).

Scenario principale:

1. L'Utente, mediante l'interfaccia *frontend*, invia una richiesta HTTP `POST` all'endpoint `/init` con un payload contenente i parametri e `dcqlQuery`.
2. Il Verifier recupera la configurazione del client dall'istanza MongoDB tramite i parametri `domainId` e `clientId`.
3. Il Verifier valida il prefisso del `clientId`.
4. Qualora il prefisso sia `openid_federation`, il Client costruisce e valida la *Trust Chain* verso il *Trust Anchor* configurato.
5. Il Verifier genera un `nonce`, un `requestId`, un `transactionId` e un `sessionId`.
6. La sessione viene persistita su Redis.
7. Il Verifier costruisce l'`authUrl`.
8. Il Verifier restituisce una risposta HTTP `201 Created` con payload `{success: true, sessionId, authUrl}`.

Scenario alternativo:

- Qualora non risulti configurato alcun Verifier per il `domainId` fornito, il sistema solleva un'eccezione. Viene restituita una risposta HTTP `403 Forbidden`.
- Qualora il `TrustAnchorService` non riesca a costruire la *Trust Chain*, il sistema solleva un'eccezione. Viene restituita una risposta HTTP `403 Forbidden`.
- Qualora non sia configurato l'url per il flusso `openid_federation`, il sistema solleva un'eccezione. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora il prefisso fornito non sia valido, il sistema solleva un'eccezione. Viene restituita una risposta HTTP `501 Not Implemented`.
- Qualora il prefisso sia `verifier_attestation` ma il token JWT non risulti valido, il sistema solleva un'eccezione. Viene restituita una risposta HTTP `400 Bad Request`.

UC1.1: Verifica dell'identità del client configurato

Attori: Verifier.
Descrizione: Il Verifier recupera dall'istanza MongoDB la configurazione del client associato ai parametri `domainId` e `clientId` ricevuti nella richiesta HTTP. Tale configurazione comprende i parametri necessari per l'intero flusso di presentazione: `redirectUris`, `vpFormatsSupported`, `jwks` e gli algoritmi crittografici supportati.
Precondizioni:

- I parametri `domainId` e `clientId` sono stati forniti nella richiesta HTTP.
- L'istanza MongoDB è raggiungibile e operativa.

Postcondizioni:

- L'oggetto `Client` è stato recuperato dall'istanza MongoDB.

Scenario principale:

1. Il Verifier recupera la configurazione del client dal repository MongoDB.
2. L'oggetto estratto dal repository viene validato e vengono verificati i campi obbligatori.

Scenario alternativo:

- Qualora nessun record corrisponda alla coppia `domainId`/`clientId` fornita, il sistema solleva un'eccezione e viene restituita una risposta HTTP `403 Forbidden`.

UC1.2: Validazione del prefisso di identificazione del client

Attori: Verifier.
Descrizione: Il Verifier effettua il *parsing* e la validazione del prefisso di identificazione del client (`prefixClientId`) al fine di determinare la modalità di autenticazione del Verifier nei confronti del Wallet.
Precondizioni:

- Il parametro `prefixClientId` è stato fornito nel corpo della richiesta HTTP; qualora risulti vuoto, viene applicato il valore predefinito `pre_registered`.
- La configurazione del client è stata recuperata con successo (rif. 2.2).

Postcondizioni:

- Il prefisso è stato ricondotto a uno dei seguenti valori: `redirect_uri`, `x509_san_dns`, `x509_hash`, `openid_federation`, `verifier_attestation`, `pre_registered`.
- Il risultato della validazione include i seguenti attributi: `clientIdValue`, `requiresSignature`, `headerExtensions`, `clientMetadataPolicy`.

Scenario principale:

1. Il Verifier effettua il *parsing* del valore del prefisso.
2. In base al prefisso riconosciuto, il sistema applica la corrispondente logica di validazione:
   - `redirect_uri`: il valore `clientIdValue` viene costruito concatenando il prefisso al `responseUri` nella forma `redirect_uri:<responseUri>`. Non è richiesta la firma crittografica del *Request Object*. La politica sui metadati del client è impostata a `MUST_INCLUDE`.
   - `x509_san_dns`: dal JWKS del client viene estratta la catena certificativa `x5c` e viene validato il certificato *leaf*. È richiesta la firma crittografica. La politica sui metadati è impostata a `MUST_INCLUDE`.

- `x509_hash`: dal JWKS viene estratta la catena `x5c` e viene calcolato il digest SHA-256 del certificato *leaf*, utilizzato come identificativo del client. È richiesta la firma crittografica. La politica sui metadati è impostata a `MUST_INCLUDE`.
- `openid_federation`: come identificativo viene adottato il valore dell'`entityIdentifier`. È richiesta la firma crittografica. I metadati del client non devono essere inclusi nella richiesta (`MUST_EXCLUDE`).
- `verifier_attestation`: viene effettuato il *parsing* del JWT di attestazione del Verifier, dal quale vengono estratti i claim `sub` e `cnf`. Il sistema verifica inoltre che il `responseUri` sia presente nella lista `redirect_uris` contenuta nel JWT. È richiesta la firma crittografica. La politica sui metadati è impostata a `MUST_INCLUDE`.
- `pre_registered`: viene utilizzato direttamente il `clientId` già registrato nel sistema. Non è richiesta la firma crittografica. L'inclusione dei metadati è facoltativa (`NOT_REQUIRED`).

Scenario alternativo:

- Qualora il prefisso fornito sia `origin` viene restituita una risposta HTTP `501 Not Implemented`.
- Per i prefissi `x509_san_dns` e `x509_hash`, qualora il JWKS del client non contenga una chiave di firma provvista di catena certificativa `x5c`, viene restituita una risposta HTTP `400 Bad Request`.
- Per il prefisso `x509_san_dns`, qualora nessun `dnsName` sia presente nel certificato corrisponda all'*hostname* del `responseUri`, viene restituita una risposta HTTP `403 Forbidden`.
- Per il prefisso `verifier_attestation`, qualora il JWT risulti privo dei claim `sub` o `cnf`, viene restituita una risposta HTTP `400 Bad Request`.
- Per il prefisso `verifier_attestation`, qualora il `responseUri` non sia incluso nella lista `redirect_uris` del JWT di attestazione, viene restituita una risposta HTTP `403 Forbidden`.

UC1.3: Validazione della Trust Chain

Attori: Verifier.
Descrizione: Qualora il prefisso di identificazione del client sia `openid_federation`, il sistema procede alla verifica della *Trust Chain* dell'OpenID Provider, preliminarmente alla generazione della sessione.
Precondizioni:

- Il prefisso identificato nel caso d'uso UC1.2 (rif. 2.2) è `openid_federation`.
- La variabile d'ambiente `OPENID4VCI_AUTHORIZE_URL` è configurata e l'endpoint corrispondente è raggiungibile.
- Almeno un *Trust Anchor* è specificato nel campo `trustedAuthorities` della *DCQL Query*.

Postcondizioni:

- L'appartenenza dell'OpenID Provider alla federazione di fiducia è stata confermata mediante la validazione completa della catena.

Scenario principale:

1. Il sistema verifica la presenza della variabile d'ambiente `OPENID4VCI_AUTHORIZE_URL` e ne estrae l'`entityId` dell'OpenID Provider.
2. Vengono estratte le `trustedAuthorities` dalla *DCQL Query* fornita dall'interfaccia *frontend*.
3. Il Verifier avvia la costruzione e la validazione della *Trust Chain* per l'`entityId` dell'OpenID Provider.

Scenario alternativo:

- Qualora la variabile d'ambiente non risulti configurata, viene restituita una risposta HTTP `400 Bad Request`.
- Qualora il `TrustAnchorService` rilevi una catena non valida, viene restituita una risposta HTTP `403 Forbidden`.

UC1.4: Creazione della sessione temporanea su Redis

Attori: Verifier.
Descrizione: Il Verifier genera l'insieme degli identificativi crittografici necessari per la sessione e ne effettua la persistenza su Redis con un TTL (*Time To Live*) configurabile.
Precondizioni:

- La validazione del prefisso è stata completata con successo (rif. 2.2).
- Qualora il prefisso sia `openid_federation`, la *Trust Chain* è stata validata (rif. 2.2).
- Il server Redis è raggiungibile e operativo.

Postcondizioni:

- Una nuova sessione con stato `pending` è stata persistita su Redis, indicizzata mediante il `requestId` come chiave primaria.
- Il TTL è stato impostato.

Scenario principale:

1. Il Verifier genera i seguenti identificativi crittografici:
   - `requestId`: utilizzato come parametro `state` nel protocollo OID4VP.
   - `nonce`: impiegato per la prevenzione degli attacchi di tipo *replay*.
   - `transactionId`: deputato al collegamento tra sessione e *Response Record*.
   - `sessionId`: restituito all'interfaccia *frontend* per il successivo recupero della presentazione.
2. Viene istanziato l'oggetto `Session` con validazione dei campi obbligatori e verifica della conformità dei valori `nonce` e `requestId`.
3. Il Client effettua la serializzazione dell'oggetto in formato JSON e ne esegue la persistenza su Redis con il TTL configurato.

Scenario alternativo:

- Qualora i valori `nonce` o `requestId` generati non superino la validazione di conformità viene restituita una risposta HTTP `400 Bad Request`.

UC1.5: Costruzione dell'Authorization URL

Attori: Verifier.
Descrizione: Il Verifier costruisce l'URL di autorizzazione (`authUrl`) destinato a essere restituito all'interfaccia *frontend* e successivamente presentato al Wallet mediante codice QR o *deep link*.
Precondizioni:

- La sessione è stata creata con successo (rif. 2.2).
- Il risultato della validazione del prefisso è disponibile (rif. 2.2).

Postcondizioni:

- L'`authUrl` è stato costruito nel formato conforme al tipo di prefisso adottato.

Scenario principale:

1. Qualora la firma crittografica non sia richiesta: l'URL viene costruito utilizzando i seguenti *query parameters*: `client_id`, `response_type=vp_token`, `response_mode`, `response_uri`, `nonce`, `state` (corrispondente al `requestId`) e `dcql_query`. Qualora la `clientMetadataPolicy` sia impostata a `MUST_INCLUDE`, viene aggiunto il parametro `client_metadata` contenente `vpFormatsSupported` e, opzionalmente, `jwks`.
2. Qualora la firma crittografica sia richiesta: l'URL viene costruito includendo esclusivamente i parametri `client_id`, `request_uri` (puntante all'endpoint `/request/:requestId`) e `request_uri_method=post`.

UC2: Generazione del Request Object

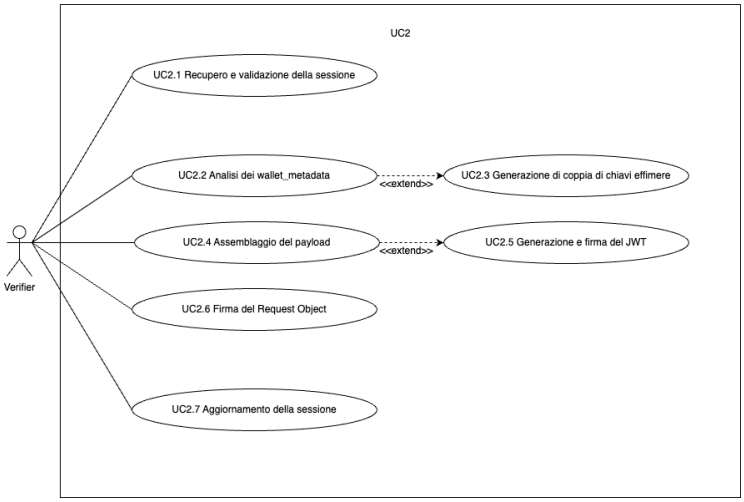


Figura 2.3: UC2 – Generazione del Request Object

Attori: Verifier, Wallet.
Descrizione: Il Wallet dell'Utente, dopo aver acquisito l'`authUrl` mediante scansione del codice QR o attivazione del *deep link*, contatta il Verifier al fine di ottenere il *Request Object* completo. Il Verifier recupera la sessione associata, negozia i parametri di sicurezza con il Wallet, quali gli algoritmi di cifratura JWE e gli algoritmi di firma, genera eventuali chiavi crittografiche effimere, assembla il *Request Object* sotto forma di JWT e lo restituisce al Wallet.
Precondizioni:

- La sessione corrispondente al `requestId` deve risultare presente su Redis e trovarsi in stato `pending`.
- Il *Request URI* non deve essere stato precedentemente consumato per la sessione in esame.
- Il client (Verifier) deve essere configurato nell'istanza MongoDB con un JWKS contenente la chiave privata di firma, qualora la firma sia richiesta dal prefisso adottato.

Postcondizioni:

- Il *Request Object* è stato generato sotto forma di JWT firmato crittograficamente.
- Qualora il Wallet supporti la cifratura JWE, una coppia di chiavi effimere è stata generata: la chiave pubblica è inclusa nel campo `client_metadata.jwks` del *Request Object*, mentre la chiave privata è stata persistita su Redis.
- La sessione è stata aggiornata in modo da impedire il riutilizzo del *Request URI*.

Scenario principale:

1. Il Wallet invia una richiesta HTTP `POST` (o `GET`) contenente, opzionalmente, i parametri `wallet_nonce` e `wallet_metadata` nel corpo della richiesta.
2. Il Verifier recupera la sessione da Redis mediante il `requestId`.
3. Il Verifier verifica che il *Request URI* non sia stato precedentemente consumato.
4. Il Verifier analizza i metadati del Wallet (`wallet_metadata`), procedendo alla negoziazione di alcuni parametri.
5. Qualora sia necessaria la cifratura JWE, il Verifier genera una coppia di chiavi effimere, persiste la chiave privata su Redis con TTL configurato e include la chiave pubblica nel campo `client_metadata.jwks`.
6. Il Verifier assembla il payload del *Request Object* conformemente alla specifica OID4VP.
7. Qualora il prefisso sia `openid_federation`, il Verifier genera e firma un *Entity Configuration* JWT con le proprie chiavi crittografiche, includendolo nel campo `trust_chain`.
8. Il Verifier appone la firma crittografica al *Request Object* mediante la propria chiave privata.
9. La sessione viene aggiornata impostando il flag `isRequestUriConsumed` al valore `true`.
10. Il Verifier restituisce il JWT con *content-type* `application/oauth-authz-req+jwt` e codice di stato HTTP `200 OK`.

Scenario alternativo:

- Qualora la sessione associata al `requestId` non risulti presente su Redis (ad esempio per scadenza del TTL), il sistema restituisce una risposta HTTP `404 Not Found`.
- Qualora il Wallet tenti di recuperare il *Request Object* per una sessione il cui *Request URI* sia stato già consumato, il sistema restituisce una risposta HTTP `403 Forbidden`. Tale meccanismo previene gli attacchi di tipo *replay*.
- Qualora il Wallet e il Verifier non dispongano di algoritmi JWE comuni, il sistema restituisce una risposta HTTP `400 Bad Request`.
- Qualora i formati *Verifiable Presentation* supportati dal Wallet non presentino alcuna intersezione con quelli del Verifier, il sistema restituisce una risposta HTTP `400 Bad Request`.
- Qualora l'algoritmo di firma indicato dal Wallet non sia compreso tra quelli supportati dal Verifier, il sistema restituisce una risposta HTTP `400 Bad Request`.
- Qualora il prefisso richieda la firma crittografica ma il JWKS del client non contenga chiavi private, il sistema restituisce una risposta HTTP `400 Bad Request`.
- Qualora la chiave privata non corrisponda al certificato *leaf* nella catena `x5c`, il sistema restituisce una risposta HTTP `400 Bad Request`.

UC2.1: Recupero e validazione della sessione

Attori: Verifier, Wallet.
Descrizione: Il Verifier recupera la sessione da Redis mediante il `requestId` fornito dal Wallet e ne verifica lo stato, accertando che il *Request URI* non sia stato precedentemente consumato. Tale verifica è finalizzata alla prevenzione di richieste duplicate e di attacchi di tipo *replay*.
Precondizioni:

- Il Wallet ha estratto il `requestId` dall'`authUrl` ricevuto.

Postcondizioni:

- La sessione è stata recuperata da Redis e risulta valida (non scaduta e non precedentemente consumata).

Scenario principale:

1. Il componente `SessionService` effettua la ricerca della sessione su Redis mediante il `requestId`.
2. Il sistema verifica che il flag `isRequestUriConsumed` risulti impostato al valore `false`.

Scenario alternativo:

- Qualora il TTL della sessione su Redis sia scaduto, il sistema solleva un'eccezione di tipo `NotFoundError`. Viene restituita una risposta HTTP `404 Not Found`.
- Qualora il flag `isRequestUriConsumed` risulti impostato al valore `true`, il sistema solleva un'eccezione di tipo `ForbiddenError`. Viene restituita una risposta HTTP `403 Forbidden`.

UC2.2: Analisi dei metadati del Wallet

Attori: Verifier.
Descrizione: Il Verifier analizza i metadati del Wallet (`wallet_metadata`) ricevuti nella richiesta HTTP `POST` e li confronta con la propria configurazione al fine di selezionare gli algoritmi crittografici comuni per la cifratura JWE della risposta, la firma del *Request Object* e i formati *Verifiable Presentation* supportati.
Precondizioni:

- La sessione risulta valida (rif. 2.2).
- La configurazione del client è stata recuperata con successo (rif. 2.2).

Postcondizioni:

- Sono stati selezionati, ove disponibili, i seguenti parametri: l'algoritmo JWE (`alg`), l'algoritmo di cifratura del contenuto (`enc`) e l'algoritmo di firma (`signingAlg`).

Scenario principale:

1. Il Verifier effettua il *parsing* dei `wallet_metadata` (da stringa JSON o oggetto strutturato).
2. Qualora i metadati risultino assenti o vuoti, vengono adottati i valori predefiniti dalla configurazione del Verifier.
3. Qualora i metadati siano presenti, il sistema procede alla ricerca dell'intersezione tra gli algoritmi supportati:
   - selezione dell'algoritmo `jweAlg`; qualora sia stato individuato un `jweAlg` ma non un corrispondente `enc`, viene tentato il *fallback* all'algoritmo `A128GCM`;
   - selezione dell'algoritmo `enc`;
   - selezione dell'algoritmo `signingAlg`.
4. Il sistema verifica la compatibilità dei formati *Verifiable Presentation*.

Scenario alternativo:

- Qualora non esista alcun algoritmo `alg` comune tra Wallet e Verifier, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora non esista alcun algoritmo `enc` comune, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora non esista alcun algoritmo di firma comune, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora non esista alcun formato *Verifiable Presentation* comune, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora sia stato negoziato esclusivamente il parametro `alg` o esclusivamente il parametro `enc` (ma non entrambi), il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.

UC2.3: Generazione della coppia di chiavi effimere

Attori: Verifier.
Descrizione: Il Verifier genera una coppia di chiavi crittografiche effimere destinate alla cifratura JWE della risposta del Wallet. La chiave pubblica viene inclusa nel *Request Object* affinché il Wallet possa cifrare la propria risposta; la chiave privata viene persistita su Redis per consentire la successiva decifratura da parte del Verifier.
Precondizioni:

- L'algoritmo JWE è stato negoziato con successo nel caso d'uso UC2.2 (rif. 2.2).
- La `clientMetadataPolicy` è impostata al valore `MUST_INCLUDE`.

Postcondizioni:

- Una coppia di chiavi effimere (pubblica/privata) è stata generata con un identificativo `kid` univoco in formato UUID.
- La chiave privata è stata persistita su Redis con un TTL configurato.
- La chiave pubblica è disponibile per l'inclusione nel *Request Object*.

Scenario principale:

1. Il Verifier determina la tipologia di chiave crittografica in funzione dell'algoritmo JWE negoziato.
2. La coppia di chiavi viene generata.
3. Le chiavi vengono esportate in formato JWK (*JSON Web Key*).
4. La chiave privata in formato JWK viene serializzata in JSON e persistita su Redis con il TTL configurato.

Scenario alternativo:

- Qualora l'algoritmo negoziato non appartenga alle famiglie ECDH o RSA, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.

UC2.4: Assemblaggio del payload del Request Object

Attori: Verifier.
Descrizione: Il Verifier assembla il payload del *Request Object* includendo tutti i parametri richiesti dalla specifica OID4VP. Il payload comprende i parametri della sessione, la *DCQL Query* e, opzionalmente, i `client_metadata`,contenenti le chiavi effimere e i formati *Verifiable Presentation* supportati e il campo `trust_chain` nel caso di OpenID Federation.
Precondizioni:

- La sessione risulta valida (rif. 2.2).
- L'analisi dei `wallet_metadata` è stata completata (rif. 2.2).
- Ove applicabile, le chiavi effimere sono state generate (rif. 2.2).

Postcondizioni:

- Il payload del *Request Object* è stato assemblato come oggetto strutturato contenente tutti i campi richiesti dal protocollo.

Scenario principale:

1. Il Verifier compone il payload includendo i seguenti claim obbligatori:
   - `iss`: identificativo del Verifier (`clientIdValue` derivato dal risultato della validazione del prefisso);
   - `aud`: destinatario (*audience*) del token;
   - `iat`: *timestamp* di emissione;
   - `exp`: *timestamp* di scadenza;
   - `client_id`: identificativo del client;
   - `response_type`: impostato al valore `vp_token`;
   - `response_mode`: modalità di risposta (`direct_post` o `direct_post.jwt`);
   - `response_uri`: URI sul quale il Wallet trasmetterà la risposta;
   - `nonce`: *nonce* crittografico della sessione;
   - `state`: corrispondente al `requestId` della sessione;
   - `dcql_query`: *DCQL Query* serializzata.
2. Il Verifier aggiunge i claim opzionali in funzione del contesto operativo:
   - `wallet_nonce`: incluso qualora il Wallet abbia fornito un *nonce* nella richiesta HTTP `POST`;
   - `client_metadata`: incluso qualora la `clientMetadataPolicy` sia impostata a `MUST_INCLUDE`.
3. Il Verifier costruisce l'header JWT con i seguenti campi: `typ` impostato a `oauth-authz-req+jwt`, `alg` corrispondente all'algoritmo di firma negoziato, e le eventuali *header extensions* specifiche del prefisso.

UC2.5: Generazione e firma del JWT dell'Entity Configuration

Attori: Verifier.
Descrizione: Qualora il prefisso di identificazione del client sia `openid_federation`, il Verifier genera un *Entity Configuration* JWT firmato con le proprie chiavi crittografiche. Tale JWT viene incluso nel campo `trust_chain` del payload del *Request Object*, consentendo al Wallet di verificare l'identità del Verifier nell'ambito della federazione di fiducia.
Precondizioni:

- Il prefisso di identificazione del client è `openid_federation`.
- La variabile `VERIFIER_ENTITY_IDENTIFIER` è configurata.
- Il JWKS del client contiene almeno una chiave privata idonea alla firma crittografica.

Postcondizioni:

- Un *Entity Configuration* JWT è stato generato e firmato crittograficamente con l'algoritmo negoziato.
- Il JWT è stato inserito nel campo `trust_chain` del payload assemblato (rif. 2.2).

Scenario principale:

1. Il Verifier estrae la chiave privata dal JWKS del client e ne deriva la corrispondente chiave pubblica.
2. Il Verifier assembla il payload dell'*Entity Configuration* conformemente alla specifica OpenID Federation.

3. Il JWT viene firmato crittograficamente con la chiave privata del Verifier.
4. Il JWT firmato viene inserito nell'array `trust_chain` del payload del *Request Object*.

Scenario alternativo:

- Qualora il JWKS non contenga alcuna chiave, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.

UC2.6: Firma del Request Object

Attori: Verifier.
Descrizione: Il Verifier appone la firma crittografica al payload assemblato (rif. 2.2) sotto forma di JWT, utilizzando la chiave privata del client e l'algoritmo negoziato. Per i prefissi basati su certificati X.509, viene preliminarmente verificata la corrispondenza tra la chiave privata e il certificato *leaf* nella catena `x5c`. Qualora la firma non sia richiesta dal prefisso, il JWT viene serializzato in modalità *unsigned*.
Precondizioni:

- Il payload del *Request Object* è stato completamente assemblato (rif. 2.2), includendo eventualmente il campo `trust_chain` (rif. 2.2).
- L'header JWT è stato costruito con i campi `typ`, `alg` e le eventuali *header extensions* specifiche del prefisso.

Postcondizioni:

- Il *Request Object* JWT è stato generato, firmato crittograficamente o in modalità *unsigned*, ed è pronto per essere restituito al Wallet.

Scenario principale:

1. Qualora la firma crittografica sia richiesta (prefissi `x509_san_dns`, `x509_hash`, `openid_federation`, `verifier_attestation`):
   - il Verifier importa la chiave privata dal JWKS del client;
   - per i prefissi `x509_san_dns` e `x509_hash`, il sistema verifica la corrispondenza tra la chiave privata e la chiave pubblica del certificato *leaf* nella catena `x5c`.
2. Qualora la firma crittografica non sia richiesta (prefissi `redirect_uri`, `pre_registered`):
   - il JWT viene serializzato in modalità *unsigned* (`alg:none`), con header e payload codificati in `base64url` e campo firma vuoto.

Scenario alternativo:

- Qualora il JWKS non contenga alcuna chiave e la firma crittografica sia richiesta, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora i parametri della chiave privata non corrispondano a quelli del certificato *leaf*, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora un prefisso non standard richieda la firma ma l'attributo `requiresSignature` risulti impostato a `false`, il sistema solleva un'eccezione di tipo `PreconditionFailedError`. Viene restituita una risposta HTTP `412 Precondition Failed`.

UC2.7: Aggiornamento della sessione

Attori: Verifier.
Descrizione: A seguito della generazione del *Request Object*, il Verifier aggiorna la sessione su Redis impostando il flag `isRequestUriConsumed` al valore `true`. Tale meccanismo garantisce che il *Request URI* possa essere utilizzato una sola volta, prevenendo attacchi di tipo *replay* e richieste duplicate da parte del Wallet.
Precondizioni:

- Il *Request Object* JWT è stato generato con successo (rif. 2.2).
- La sessione risulta presente su Redis.

Postcondizioni:

- Il flag `isRequestUriConsumed` della sessione è stato impostato al valore `true` su Redis.
- Qualsiasi richiesta successiva recante il medesimo `requestId` verrà rifiutata dal sistema.

Scenario principale:

1. Il Verifier aggiorna la sessione su Redis impostando il flag `isRequestUriConsumed` al valore `true`.
2. Il Verifier restituisce il *Request Object* JWT al Wallet nella risposta HTTP con codice di stato `200 OK`.

2.3 Limiti

Capitolo 3

Progettazione

In questo capitolo verrà trattato il secondo periodo del tirocinio: la progettazione e la codifica. Questa parte, avvenuta dopo l'Analisi dei Requisiti, è da ritenersi importante per il progetto in quanto ha occupato gran parte del tempo lavorativo. Il primo passo è stato eseguire la progettazione dell'architettura, in modo che i servizi necessari fossero correttamente configurati prima del loro effettivo utilizzo. Successivamente è stata eseguita la stesura del codice per la realizzazione della Skill. Nei paragrafi successivi seguiranno porzioni di codice significativo per la loro importanza e per il funzionamento di determinati servizi. Importante ricordare che il codice implementato e mostrato sarà in Node.js come quanto detto nel paragrafo 1.4.1.

3.1 Architettura

Come riportato nel paragrafo 1.4.3, il progetto si basa interamente sui servizi offerti dall'ecosistema Amazon. Di conseguenza è risultato facile ed immediato impostare i servizi dell'architettura in modo da rendere il tutto efficacie ed efficiente. In questo paragrafo si andrà a spiegare cosa offre ogni singolo servizio di AWS e come esso è stato impostato per il suo corretto funzionamento nella Skill.

Capitolo 4

Codifica

Dopo aver completato la fase di progettazione è stata eseguita la fase di codifica e validazione. Questo capitolo sarà il conseguimento della seconda parte del periodo di stage dove verranno spiegate le funzionalità implementate nella Skill mostrando porzioni di codice. Verrà inoltro spiegata la struttura del progetto: le varie cartelle contenti i file delle *classi* realizzate e i *packages* installati.

4.1 Organizzazione del codice

Successivamente alla parte di comprensione e configurazione dei servizi utilizzati in questo progetto, e alla creazione della Skill, è stato eseguito un micro-processo di organizzazione del codice prima della sua stesura. Questa pratica adottata è stata necessaria per avere uno schema e una gerarchia delle parti di codice che sono state sviluppate durante il periodo di tirocinio. Tale organizzazione ha permesso di sviluppare in maniera più ordinata e consapevole riducendo la probabilità di introdurre errori e rendendo il codice più manutenibile. Pertanto la cartella del progetto è stata così disposta:

Capitolo 5

Verifica e validazione

L'ultima fase del periodo di tirocinio ha riguardato il processo di verifica e validazione. La prima è un processo che si occupa di fornire evidenza oggettiva, ovvero che i risultati ottenuti come output dello sviluppo del software soddisfino i requisiti, la seconda invece è un processo per confermare in modo definitivo che le caratteristiche del software siano conformi ai bisogni dell'utente e all'uso previsto. Per eseguire la verifica e la validazione è necessario l'esecuzione di test[1] che individuino carenze, correttezza e affidabilità. Nel progetto della Skill Concierge Croccante non è stato realizzato alcun test per il processo di verifica: viste le tempistiche non è stato potuto fare ciò e si è pertanto scelto di eseguire test già esistenti nel servizio Amazon Lambda, sufficienti ad esaminare il codice e rilevare eventuali errori. Per quanto riguarda il processo di validazione[2] è stato svolto, alla fine del periodo di tirocinio, un incontro con il tutor aziendale e CFO Damiano Buscemi come conferma fiale la quale ha accertato le bontà richieste dal prodotto finale. Tale validazione non è stata altro che una dimostrazione funzionante della Skill, eseguita prima da me per spiegare le varie funzionalità, ed infine da Crispy Bacon come conferma.

[1] In ambito informatico i test eseguiti nel processo di verifica sono eseguiti in maniera automatica durante lo sviluppo del software

[2] Attività di supporto la quale accerta che il prodotto dei processi rispetti le specifiche

Capitolo 6

Valutazione finale

In questo ultimo capitolo verranno fatte le analisi dei risultati ottenuti, dei dati e delle informazioni raccolte ed emerse durante il tirocinio e una valutazione personale di come è stata vissuta tale esperienza. Verranno fatti anche dei commenti critici sul risultato del prodotto ottenuto, sul lavoro svolto e sugli strumenti utilizzati. Infine verranno fatte delle osservazioni sulla possibilità di miglioramenti sul prodotto, sulla metodologia di lavoro e possibilità di estensioni del codice.

Il progetto di stage aveva come obiettivo la realizzazione di una Skill, pensata per Amazon Echo Show, per l'accoglienza di visitatori utilizzando le funzionalità e i servizi offerti da Amazon Web Services. Lo scopo principale del tirocinio era la sperimentazione con le tecnologie Node.js ed Alexa Presentation Language per realizzare un prototipo di prodotto su cui individuare le potenzialità e i limiti, confrontandole anche con tecnologie simili come Google Assistant. In tale contesto, ad inizio stage assieme al tutor aziendale sono stati definiti gli obiettivi minimi e massimi da portare a termine nell'arco delle 300-320 ore a disposizione. Gli obiettivi minimi racchiudono tutte le attività incentrate sullo studio ,sull'utilizzo delle tecnologie AWS e l'integrazione di servizi terzi finalizzati al mio stage. Gli obiettivi massimi, invece, riguardavano il soddisfacimento dei requisiti desiderabili quale la registrazione della durata degli appuntamenti determinati dall'inizio e dalla fine dell'accoglienza fisica fatta dal personale.

6.1 Risultati ottenuti

6.2 Analisi critica del prodotto

Analizzando criticamente il prodotto si può affermare che la Skill è stata sviluppata con una buona base conversazionale semplice e non eccessivamente lunga da svolgere, fattore rilevante visto il contesto degli assistenti vocali. Altro valore aggiunto è il fatto di aver realizzato un prodotto con funzionalità ben definite, così da non causare smarrimento all'utente durante l'utilizzo.

6.3 Analisi critica del lavoro

Nel corso dello stage ho avuto la possibilità di mettere alla prova le mie competenze di organizzazione del lavoro e pianificazione delle attività per la prima volta al di fuori dell'ambito universitario, competenze acquisite per la maggior parte dal corso di Ingegneria

del Software[1]. Nonostante io sia stato seguito ed aiutato dal mio tutor aziendale, la gestione temporale del progetto è stata per la maggior parte affidata a me. L'interazione con il tutor in questo contesto è stata principalmente espressa da incontri per ricevere feedback sull'organizzazione delle attività e sullo stato dei lavori. Questo mi ha permesso di maturare in maniera autonoma la priorità delle varie fasi, migliorando così le mie capacità organizzative grazie all'esperienza diretta. Altra soddisfazione ottenuta è data dall'esperienza acquisita utilizzando i servizi offerti dall'ecosistema Amazon Web Service con il quale è stato progettato il prodotto finale, molto apprezzato visto il numeroso ambito d'uso. Complessivamente sono molto soddisfatto dell'esperienza di stage svolta presso Crispy Bacon spa: oltre all'ottimo ambiente di lavoro e del personale, riscontrato in azienda, ho avuto modo di accrescere la mia figura professionale. Alla luce delle conoscenze e competenze acquisite durante il tirocinio posso affermare con sicurezza che le mie aspettative personali sono state completamente corrisposte.

[1]SWE. URL: https://www.math.unipd.it/ tullio/IS-1/2018/

6.4 Valutazione degli strumenti utilizzati

Per quanto riguarda gli strumenti utilizzati ho potuto mettere in pratica, in maniera adattata alle esigenze dell'azienda, il modello di sviluppo agile Scrum, fin'ora solamente affrontato dal punto di vista teorico all'interno del corso di Ingegneria del Software. Mettere in pratica questo modello all'interno di un contesto aziendale mi ha permesso di comprenderne i benefici e gli svantaggi che ne consegue il suo utilizzo.
È stato inoltre molto apprezzato l'uso della suite di Amazon Web Sevice con il quale ho potuto mettermi alla prova integrando i principali servizi utilizzati come DynamoDB e SES. Il loro utilizzo infatti mi ha permesso di approfondire meglio le loro potenzialità, imparando così a comprendere l'ampio campo d'uso che queste offrono. In aggiunta utilizzando l'ambiente di sviluppo Visual Studio Code ho potuto procedere alla stesura del codice in maniera agevolata, questo grazie alla possibilità di estensione dell'IDE con plugin per supportare al meglio i linguaggi di programmazione.

Bibliografia